

Relatório de Design e Decisões Arquiteturais - Projeto Jackut

João Victor Duarte do Nascimento

Arquitetura e Fluxo de Dados (Persistência em Memória)

O projeto foi desenhado para não depender de um banco de dados relacional tradicional. A arquitetura foca em alta performance e simplicidade através de um fluxo em memória (In-Memory). Enquanto o sistema está em execução, todos os usuários e seus relacionamentos são armazenados na RAM através de Coleções Nativas do Java (especificamente *Map* e *List*).

Para garantir a persistência dos dados entre as execuções (requisito vital para os testes de aceitação da User Story 1 e 2), foi adotado o uso de arquivos XML. Ao iniciar, a Facade utiliza a classe `java.beans.XMLDecoder` para remontar o estado do sistema. No encerramento (`encerrarSistema()`), a classe `java.beans.XMLEncoder` serializa as coleções da RAM em um arquivo *banco_jackut.xml*. **Este arquivo funciona como o principal artefato de saída da persistência do projeto, atuando como o "banco de dados" físico que preserva o estado exato das coleções estruturadas para execuções futuras.** Para que essa serialização ocorra sem erros, a entidade principal foi projetada com construtores vazios e métodos Getters/Setters completos.

Padrões de Projeto Utilizados (Design Patterns)

- **Facade** (Fachada): O núcleo da comunicação com a biblioteca de testes *EasyAccept* baseia-se no padrão Facade. A classe *Facade.java* atua como uma interface unificada que expõe apenas os métodos estritamente necessários (com as assinaturas exatas exigidas pelos scripts *.txt*). Decisão de Design: A Facade não possui nenhuma regra de negócio. Seu papel é atuar como um escudo de baixo acoplamento, delegando todas as chamadas para os controladores adequados e gerenciando o ciclo de vida da persistência (XML).

- **Controller** (Controlador): Separamos a inteligência do sistema na classe *ControladorDeUsuarios*. Isso garante Alta Coesão, pois a entidade *Usuario* fica responsável apenas por guardar estado (dados), enquanto o Controlador centraliza as regras de segurança (verificação de senhas), lógica de relacionamentos (mão dupla de amizades) e a fila de mensagens (FIFO).

Estrutura de Diretórios

Para manter o projeto organizado e separar as responsabilidades, os arquivos foram divididos nos seguintes pacotes lógicos:

- */*: Raiz do projeto contendo as portas de entrada (*Main.java* e *Facade.java*).
- */models*: Contém a classe de entidade pura (POJO), representando o estado dos dados.
- */controllers*: Acomoda as classes que processam as regras de negócio.
- */exceptions*: Isola o tratamento de erros em exceções personalizadas para que o *EasyAccept* capture as mensagens exatas exigidas.

Modelagem da Entidade (Alta Flexibilidade)

Diferente de sistemas rígidos, no Jackut existe apenas uma classe concreta: *Usuario*. Como a rede social exige que os perfis sejam moldados dinamicamente pelos próprios usuários (User Story 2), a decisão de design foi abolir atributos fixos (como *estadoCivil* ou *cidadeNatal*) e implementar um *Map<String, String> perfil*. Isso permite expansão infinita de atributos sem necessidade de recompilar a classe. As conexões lógicas (amizades, convites e recados) são mapeadas em estruturas *List<String>*, garantindo o rastreamento cronológico das interações (como a fila de recados).

A decisão de design central para esta US foi tratar a amizade como uma transação de mão dupla obrigatória. Para evitar complexidade com entidades intermediárias, utilizou-se o rastreamento via listas no próprio modelo. Quando um usuário solicita amizade, o alvo vai para a lista de *convitesEnviados*. A amizade só é efetivada (movida para a lista *amigos*) quando o alvo realiza a ação de retorno, garantindo o consentimento mútuo exigido pelos testes.

A decisão adotada para garantir a leitura cronológica das mensagens foi simular o comportamento de uma Fila (FIFO - First In, First Out) utilizando uma *List<String>* nativa. Os recados são adicionados no final da lista do destinatário, e o método *lerRecado* foi

projetado para sempre consumir e remover o elemento de índice 0 da memória, otimizando o fluxo sem precisar de ordenação adicional.

PRIMEIRA FASE DO PROJETO - USUÁRIOS E REDE SOCIAL

Toda a responsabilidade de gerenciar os dados está centralizada na classe *ControladorDeUsuarios*, que manipula a coleção principal do sistema: um *Map<String, Usuario>*, onde a chave de busca (ID único) é o próprio *login* do usuário.

Diferente de sistemas com múltiplas hierarquias, no Jackut existe apenas uma classe concreta *Usuario*. Ela concentra todas as informações necessárias para a rede social: credenciais (login, senha e nome), um mapa dinâmico para o perfil (permitindo a criação de atributos customizados pelo próprio usuário) e listas (*List<String>*) para gerenciar as conexões (amigos, convites enviados e recados recebidos).

Ponto importante: Como estamos usando o decodificador para XML (XMLEncoder/XMLDecoder), é obrigatório que a classe *Usuario* possua um construtor vazio, além de métodos *getters* e *setters* para absolutamente todas as coleções e atributos. Sem isso, o Java não consegue remontar o objeto ao ler o arquivo salvo em disco.

Na pasta */controllers*, a classe *ControladorDeUsuarios* concentra a lógica de negócio e as validações para garantir a integridade da rede. Os métodos mais importantes incluem:

- **zerar:** Limpa o mapa de usuários, resetando o sistema.
- **criarUsuario:** Válida se os dados base não são nulos/vazios e se o login já está em uso antes de instanciar a conta.
- **abrirSessao:** Sistema de autenticação (login/senha) que, por segurança, lança uma exceção genérica em caso de erro, evitando expor se a falha foi no login ou na senha.
- **editarPerfil:** Adiciona ou modifica dinamicamente atributos no mapa de perfil do usuário.
- **adicionarAmigo / ehAmigo:** Controla o fluxo duplo de amizade (um usuário envia o convite, e a amizade só é efetivada quando o alvo adiciona de volta).
- **enviarRecado / lerRecado:** Gerencia a fila (FIFO) de mensagens diretas entre os usuários.

Diagrama de Classes

Abaixo está o diagrama simplificado ilustrando o fluxo de delegação e composição do sistema:

